A MACHINE LEARNING APPROACH TO EMPLOYEE PERFORMANCE PREDICTION

A project Report Submitted in Partial fulfillment of the

Requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by:

RELLU BHANUSATYASRI – 22T91A0403

GUNISETTI RAMYA – 22T91A0413

MUTHYALA SUDHEER – 22T91A0428

KOPPULA CHAITANYA – 22T91A0422

DANABOINA NAGA TEJA YADAV – 21T91A0408

Under the guidance of

Mr. P. Siva Krishna, M. Tech (Ph.D)

Assistant Professor

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

GIET ENGINEERING COLLEGE

(Approved by A.I.C.T E, Affiliated to J.N.T.U, Kakinada)

NH-16, Chaitanya Knowledge City, Rajahmundry - 533296

April - 2026

GIET ENGINEERING COLLEGE

NH-16, Chaitanya Knowledge City, Rajahmundry-533296

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that the project work entitled "**A Machine Learning Approach to Employee Performance Prediction"** is being submitted for partial fulfillment of BACHELOR OF TECHNOLOGY in ELECTRONICS AND COMMUNICATION ENGINEERING to GIET ENGINEERING COLLEGE, Rajahmundry. AP affiliated to the J.N.T.U.Kakinada, is Bonafide work done by RELLU BHANUSATYASRI Bearing Roll No. 22T91A0403, GUNISETTI RAMYA Bearing Roll No: 22T91A0413, MUTHYALA SUDHEER Bearing Roll No 22T91A0428, KOPPULA CHAITANYA Bearing Roll No 22T91A0422, DANABOINA NAGA TEJA YADAV Bearing Roll No 21T91A0408 under our guidance during the academic year 2025-2026 and it has been found suitable for acceptance according to the requirement of University

These results embodied in the project report have not been submitted to any other university or institute for the award of degree

Mr. P. Siva Krishna, M. Tech (Ph.D)
Internal Guide
Department of ECE

Dr. T. Srinivas, M.Tech, MIETE, Ph.D
Head Of the Department
Department of ECE

External Examiner

ACKNOWLEDGMENT

We are grateful to our guide Mr. P.SIVA KRISHNA, M.Tech,(Ph.D) Assistant Professor, for giving us the opportunity to carry out this project work. We take this opportunity to express our profound and whole heartful thanks to our guide, who with his patience support and sincere guidance helped us in successful completion of the project. We are particularly indebted to him for his innovative ideas, valuable suggestions and guidance during the entire period of our project work and without his unfathomable energy and enthusiasm, this project would not have been completed.

We would like to express our deep sense of gratitude to Dr. M.V.S.BABU, M.E, Ph.D Principal, for providing us a chance to under go the course in the prestigious institute. We would like thank Dr. T.SRINIVAS, M. Tech, MIETE, Ph.D. Associate Professor and Head of the Department of ECE, for valuable suggestions throughout our project which have helped in giving definite shape to this work.

We would like to thank Mr. A.G.V.KARTHIK RAJU, Assistant Professor, Mrs. B.HEMA LATHA, Assistant Professor, Mr. P.SIVAKRISHNA, Assistant Professor, Mr.CH. RAJASEKHARA REDDY, Assistant Professor, Mr. B. VENKATESH, Assistant Professor, Mr. O.SUDHAKAR, Assistant Professor, Mrs. G SRUJANA, Assistant Professor, Mrs. K V KALYANI, Assistant Professor for valuable suggestion throughout our project which have helped in giving definite shape to this work. Finally, we would like to thank all the faculty members and nonteaching staff of Department of Electronics and Communication Engineering, GIET ENGINEERING COLLEGE for their direct and indirect help during the project.

We own our special thanks to the MANAGEMENT of our college for providing necessary arrangements to carry out the project.

The euphoria and satisfaction of completion this project will not be completed until we thank all the people who have helped us in the successful completion of this enthusiastic task.

Lastly, we thank our parents for their ever-kind blessings.

RELLU BHANUSATYASRI	-	22T91A0403
GUNISETTI RAMYA	-	22T91A0413
MUTHYALA SUDHEER	-	22T91A0428
KOPPULA CHAITANYA	-	22T91A0422
DANABOINA NAGA TEJA YADAV	-	21T91A0408

ABSTRACT

Employee performance plays a crucial role in determining the overall productivity and success of an organization. In recent years, organizations have increasingly relied on data-driven techniques to analyze workforce behavior and predict performance trends in advance. This project aims to design and implement a supervised machine learning–based predictive model to forecast employee performance levels using historical human resource data. The proposed system focuses on building a classification model that can categorize employees into performance classes such as active or underperforming, based on selected employee attributes.

The project will involve data collection from an existing employee dataset, followed by essential data preprocessing steps, including data cleaning, handling missing values, feature selection, and data normalization. Multiple supervised learning algorithms such as K-Nearest Neighbors (KNN), Decision Tree, Logistic Regression, and Support Vector Machine (SVM) will be trained and evaluated to determine the most suitable model for employee performance prediction. The dataset will be split into training and testing sets to analyze model behaviour and generalization capability.

Performance evaluation will be carried out using standard metrics such as accuracy, precision, recall, F1-score, and confusion matrix, ensuring reliable assessment of each supervised algorithm. The expected outcome of this project is a well-trained machine learning model capable of predicting employee performance with high accuracy, which can assist organizations in identifying performance trends and supporting data-driven decision-making. This project is purely a software-based machine learning model and does not involve application development or hardware components.

TABLE OF CONTENTS

CHAPTER NO	CONTENTS	Page No.
	ABSTRACT	**I**
	TABLE OF CONTENTS	**II**
	LIST OF FIGURES	**VI**
	LIST OF TABLES	**VII**
	LIST OF ABBREVATIONS	**VIII**
1	**INTRODUTION**	1
1.1	Project Introduction	1
1.2	Problem Statement	2
1.3	Objectives of the Project	2
1.4	Scope of the Project	3
1.5	Literature Survey	4
2	**SYSTEM ANALYSIS**	5
2.1	Existing system	5
2.1	Disadvantages of Existing System	5
2.2	Proposed System	7
2.2.1	Advantages of Proposed system	8
2.3	Feasibility Study	8
2.3.1	Economic Feasibility	8
2.3.2	Technical Feasibility	9
2.3.3	Operational feasibility	9
3	**REQUIREMENTS SPECIFICATION**	10

3.1	Hardware Requirements	10
3.2	Software Requirements	10
3.3	Functional Requirements	11
3.4	Non-Functional Requirements	12
3.5	Dataset Requirements	13
4	**TECHNOLOGIES USED**	14
4.1	Python	14
4.2	VS Code / Jupyter Notebook	14
4.3	Flask	15
4.4	HTML	16
4.5	CSS	17
4.6	JavaScript	17
4.7	Numpy	18
4.8	Pandas	18
4.9	Scikit-learn	19
4.10	XGBoost	19
4.11	Matplotlib / Seaborn	19
4.12	Logistic Regression	20
4.13	Random Forest Classifier	20
4.14	Train-test split	20
4.15	Label Encoding / One-Hot Encoding	21
5	**SYSTEM DESIGN**	22
5.1	System Architecture	22
5.2	Data Processing	23
5.3	Machine Learning Model Design	24

5.4	Web Application Design	25
5.5	Input and Output Flow	25
6	**CODING AND IMPLEMEMNTATION**	27
6.1	Importing Packages	27
6.2	Data Loding	28
6.3	Data preprocessing	28
6.4	Feature Encoding	29
6.5	Model Building	29
6.6	Model Training	30
6.7	Model Evaluation	30
6.8	Flask Integration	31
6.9	Frontend Implementation	32
7	**SYSTEM TESTING**	33
7.1	Introduction to testing	33
7.2	Testing and Validation	33
7.3	Unit Testing	34
7.4	Integration Testing	34
7.5	Functional Testing	34
7.6	System Testing	35
7.7	Black Box Testing	35
7.8	Test Cases and Validation Result	36
8	**SCREENSHOTS**	37
9	**CONCLUSION AND FUTURE ENHANCEMENT**	41

9.1	Conclusion	41
9.2	Future Enhancement	41
10	**REFERENCES**	43
10.1	Research Papers	43
10.2	Websites	43
	ANNEXURE	
A.1	Frontend Implementation	45
A.2	Backend Implementation	48
A.3	Machine Learning Model Code	51
A.4	Model Saving & Integration	53

LIST OF FIGURES

Sl. No.	Figure No.	Name of the Figure	Page No.
1	8.1	Title and Description Page	44
2	8.2	Single Employee Input Form	44
3	8.3	Single Prediction Output Page	45
4	8.4	Multiple Employee Input Page	46
5	8.5	Multiple Employee Prediction Result Page	46
6	8.6	Flask Backend Output	47

LIST OF TABLES

Sl. No.	Table No.	Name of the Table	Page No.
1	3.1	Hardware Requirements	14
2	3.2	Software Requirements	14
3	7.1	Test Cases and Validation Results	54

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ML	Machine Learning
HR	Human Resource
AI	Artificial Intelligence
LR	Logistic Regression
RF	Random Forest
XGBoost	Extreme Gradient Boosting
CSV	Comma Separated Values
UI	User Interface
API	Application Programming Interface
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
IDE	Integrated Development Environment
EDA	Exploratory Data Analysis
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
KPI	Key Performance Indicator

CHAPTER 1

INTRODUTION

1.1 Project Introduction

In the modern corporate world, employee performance is one of the most significant factors that directly influence productivity, profitability, service quality, and overall organizational growth. Every organization, regardless of its size, depends on the quality of work delivered by its employees. Therefore, measuring and improving employee performance has become a crucial responsibility for management and human resource departments.

Traditionally, employee performance evaluation is carried out using manual appraisals, supervisor feedback, periodic reviews, and performance reports. Although these methods are widely used, they often involve several limitations. Manual evaluations may consume considerable time, depend heavily on subjective judgement, and sometimes produce inconsistent or delayed decisions. In many cases, human bias or incomplete analysis may affect the fairness and accuracy of the evaluation process.

To overcome these limitations, organizations are increasingly moving towards data-driven decision-making. Machine Learning (ML) offers an effective solution for this need by identifying patterns in historical data and making intelligent predictions. In employee analytics, machine learning can be used to predict performance outcomes based on past records and employee-related attributes. This enables organizations to take proactive steps such as employee training, role alignment, motivation planning, and performance improvement strategies.

The proposed project, “A Machine Learning Approach to Employee Performance Prediction,” is designed to develop a predictive system that classifies employee performance using historical HR-related data. The system uses important features such as employee age, experience, education level, salary, training score, attendance, number of projects, overtime, job satisfaction, and related factors to determine the likely performance category of an employee.

This project uses three supervised machine learning classification algorithms: Logistic Regression, Random Forest Classifier, and XGBoost Classifier. These algorithms are chosen because they are effective for classification tasks and provide reliable performance in predictive modelling. After training and testing the models, the best-performing model is selected and integrated into a web application using Flask. The final application allows users to enter employee details and instantly view the predicted performance level.

1.2 Problem Statement

Employee performance evaluation is often performed manually in many organizations. Traditional evaluation systems rely on supervisor observations, appraisal forms, and periodic performance reviews. These methods are time-consuming, may lack consistency, and can sometimes be influenced by personal bias or incomplete data analysis. Because of this, organizations may fail to identify performance trends early, resulting in poor workforce planning and delayed decision-making.

The problem addressed in this project is the lack of an automated, objective, and intelligent system that can predict employee performance based on historical employee data. There is a need for a system that can analyse multiple employee-related factors and provide accurate performance predictions quickly and reliably.

This project aims to solve that problem by building a machine learning-based employee performance prediction system that can classify employee performance levels and assist management in making better HR decisions.

1.3 Objectives of the Project

The main objectives of this project are:

1. To develop a machine learning-based system for predicting employee.
2. To collect and preprocess employee-related historical data for model training.
3. To identify important factors that influence employee performance.
4. To implement and compare Logistic Regression, Random Forest Classifier, and XG Boost Classifier.

5. To evaluate model performance using classification metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
6. To select the best-performing model for deployment.
7. To build a web-based interface using Flask, HTML, CSS, and JavaScript for real-time prediction.
8. To support HR analytics and data-driven workforce management.

1.4 Scope of the Project

The scope of this project includes:

- Developing a software-based machine learning model for employee performance prediction.
- Using a structured employee dataset in CSV format.
- Performing data preprocessing such as cleaning, encoding, and feature
- Training classification models using supervised learning.
- Comparing the performance of multiple classifiers.
- Deploying the best model in a Flask web application.
- Accepting manual user input through a frontend form.
- Displaying the predicted employee performance level instantly.

This project does not involve hardware integration, IoT devices, biometric sensors, or live enterprise HR database integration. It is primarily a machine learning and web application-based academic project.

1.5 Literature Survey

Employee performance prediction has become an important area in human resource analytics due to the increasing need for data-driven employee management. Several researchers have explored the use of machine learning algorithms to automate performance analysis and improve the reliability of employee evaluation systems.

Early studies focused on replacing manual appraisal methods with classification-based models. Researchers observed that traditional performance review systems were often subjective and lacked consistency. Machine learning methods were proposed to reduce human bias and improve prediction quality.

A study by Kirimi and Moturi (2016) demonstrated that classification techniques can be used effectively for employee performance prediction. Their work highlighted the importance of selecting meaningful employee attributes and using data driven methods for decision support.

Lather et al. (2019) compared multiple supervised machine learning algorithms for work-force related prediction tasks. Their findings suggested that ensemble models often perform better than simple baseline models when feature relationships are complex.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Existing System

Many organizations, employee performance is evaluated using traditional appraisal methods. These In include:

- Manual performance review forms
- Supervisor-based feedback
- Attendance and behaviour observation
- Periodic performance appraisals
- Managerial ranking systems

These methods depend largely on human judgement and historical review practices. Although they are useful, they may not be sufficient for analysing large amounts of employee data or identifying hidden patterns in performance.

2.1.1 Disadvantages of Existing System

The existing system has several limitations:

1. Time-consuming process

- Manual reviews take significant time.
- Requires continuous manual monitoring and evaluation.
- Large amount of paperwork increases workload.
- Slows down decision-making in organizations.
- Repetitive tasks reduce overall efficiency.
- Difficult to track performance regularly.

2. Subjective judgement

- Decisions may vary from one evaluator to another.
- Decisions depend on individual opinions.
- Lack of standardized evaluation criteria.
- Different evaluators may give different ratings.
- Personal preferences may affect outcomes.
- Reduces fairness in employee assessment.

3. Human bias

- Personal opinions may influence performance ratings.
- Favoritism may influence performance ratings.
- Gender, background, or personal relations may impact decisions.
- Leads to unfair evaluation of employees.
- Reduces trust in the system.

4. Limited scalability

- Difficult to manage for large organizations.
- Difficult to handle large employee data manually.
- System becomes inefficient as organization grows.

5. Delayed decisions

- Performance issues are identified late.
- Slower response to employee improvement needs.

- Affects productivity and growth.
- Delays promotions or corrective actions.

6. Low analytical capability

- Traditional systems cannot identify complex patterns.
- Cannot process large datasets effectively.
- Limited use of historical data.
- Decision-making lacks data-driven insights

2.2 Proposed System

The proposed system is a machine learning-based employee performance prediction system. It uses historical employee data to classify employee performance automatically. The system processes input features such as employee demographics, work-related attributes, training details, attendance, and job satisfaction to predict performance level.

The proposed system includes:

- Dataset loading
- Data preprocessing
- Feature preparation
- Model training using
  - Logistic Regression
  - Random Forest Classifier
  - XGBoost Classifier
- Model evaluation and comparison
- Best model selection
- Flask-based deployment

- User input form and prediction output

2.2.1 Advantages of Proposed System

The proposed system offers many advantages:

1. Faster prediction – Reduces manual evaluation time.
2. Data-driven decisions – Uses actual employee data instead of only opinions.
3. Improved accuracy – Machine learning identifies hidden patterns.
4. Reduced bias – Minimizes dependency on subjective judgement.
5. Scalability – Can handle larger datasets efficiently.
6. Early identification – Detects likely performance outcomes in advance.
7. Practical deployment – Web application makes it easy to use.
8. Supports HR planning – Helps in training, promotions, and role assignment.

2.3 Feasibility Study

A feasibility study determines whether the project is practical, economical, and technically achievable. The proposed system is analysed under three major aspects:

2.3.1 Economic Feasibility

The project is economically feasible because it uses mostly open-source tools and free soft-ware. The cost of development is minimal.

Cost Estimation:

- Software Cost: Free
- Dataset Cost: Free / Publicly Available
- Development Tools: Free

- Hardware Cost: Existing system used
- Maintenance Cost: Minimal
- Total Estimated Cost: ₹0 (Zero Cost Project)

2.3.2 Technical Feasibility

The project is technically feasible because the required technologies are widely available and easy to implement. Python provides strong support for machine learning. Flask allows easy deployment of models into a web interface. The system can run on a normal lap-top or desktop without any special hardware. Technical feasibility is supported by:

- Python ecosystem for ML
- Scikit-learn for Logistic Regression and Random Forest
- XGBoost library for boosting-based classification
- Pandas and NumPy for data handling
- Flask for backend integration
- HTML/CSS/JavaScript for frontend development

2.3.3 Operational / Social Feasibility

The project is operationally feasible because it is easy to use and provides useful predictions for HR teams and management. Users can manually enter employee data through a web form and receive instant results.

The system is socially beneficial because

- It improves fairness in performance evaluation
- It supports employee development decisions

CHAPTER 3

REQUIREMENTS SPECIFICATION

3.1 Hardware Requirements

The system can run on a normal computer with the following minimum configuration. These requirements ensure that the application runs smoothly and efficiently without performance issues.

Component	Requirement
Opera ng System	Windows 10 / 11
Processor	Intel i5 or above
RAM	8 GB or above
Storage	10 GB free space

Table 3.1: Hardware Requirements

3.2 Software Requirements

Software / Tool	Purpose
Python	Core programming language
Jupyter Notebook / VS Code	Development environment
Flask	Backend web framework
HTML	Frontend structure
CSS	Frontend styling

Table3.2: Software Requirements

3.3 Functional Requirements

The system should be able to: Load the employee dataset-The system must allow loading of the employee dataset from a file (such as CSV or Excel). This dataset contains all necessary employee related information required for analysis and prediction

1.Clean and preprocess the data-The system should handle missing values, remove duplicates, and correct inconsistencies in the dataset to ensure data quality and accuracy

2.Remove irrelevant columns-Unnecessary or unused columns that do not contribute to prediction should be identified and removed to improve model efficiency and reduce complexity

3.Encode categorical values-Categorical data such as gender, department, or job role should be converted into numerical format using encoding techniques so that ma-chine learning models can process them.

4.Split data into training and testing sets-The system should divide the dataset into training and testing parts, usually in a fixed ratio (e.g., 80:20), to train the model and evaluate its performance.

5.Train Logistic Regression model. The system should train a Logistic Regression model using the training data to perform classification and predict employee performance

6.Train Random Forest Classifier model-The system should also train a Random Forest Classifier, which uses multiple decision trees to improve prediction accuracy and reduce overfitting.

7.Train XGBoost Classifier model-An advanced model, XGBoost Classifier, should be trained to achieve high performance using gradient boosting techniques.

8.Compare model performance -The system must evaluate all trained models using metrics like accuracy, precision, recall, or F1-score to compare their effectiveness.

9.Select the best performing model -Based on the comparison results, the system should automatically select the model with the highest performance for final predictions

10.Accept user input through a web form- The system should provide a user-friendly web interface where users can enter employee details required for prediction

11.Predict employee performance based on entered values-Using the selected model, the system should analyze the user input and generate a prediction about employee performance.

3.4 Non-Functional Requirements

The system should satisfy the following non-functional requirements:

•**Accuracy** – Predictions should be reasonably reliable the system must provide accurate and dependable predictions based on the trained machine learning models. Proper data preprocessing and model selection should be ensured to improve prediction quality and minimize errors.

- **Usability** – Interface should be simple and user-friendly the system should have an easy to use interface so that even non-technical users can operate it without difficulty. Clear instructions, simple design, and well-organized input fields should be provided.

- **Performance** – Results should be generated quickly the system should process input data and generate predictions within a short time. Efficient algorithms and optimized code should be used to ensure fast response and smooth operation.

- **Scalability** – Can support larger datasets in future the system should be designed in a way that it can handle increasing amounts of data without affecting performance. It should be capable of scaling up when more employee data is added.

- **Maintainability** – Code should be modular and easy to update the code should be written in a clean and structured manner using modular programming practices. This will make it easier to fix bugs, update features, and improve the system in the future.

- **Reliability** – System should work consistently the system should perform its functions correctly every time without failures. It should handle errors properly and ensure stable operation under different conditions.

- **Portability** – Can be run on common systems with Python installed the system should be compatible with different environments and should run on any system that has Python installed.

3.5 Dataset Requirements

The project requires a structured employee dataset in CSV format containing relevant employee attributes and a target performance label.

Example features used in implementation:

- Age
- Gender
- Education Level
- Job Role
- Department
- Years of Experience
- Work Hours
- Attendance Percentage
- Number of Projects
- Overtime
- Job Satisfaction

Dataset Characteristics:

- Mixed numerical and categorical features
- Historical employee records
- Target variable for classification

CHAPTER 4

TECHNOLOGIES USED

4.1 Python

Python is the main programming language used in this project. It is simple, readable, and highly suitable for machine learning and data analysis tasks. Python provides a large ecosystem of libraries for preprocessing, model building, visualization, and deployment. Its easy syntax and vast community support make it one of the most preferred languages for developing intelligent systems.

Python also supports multiple programming paradigms such as object-oriented, procedural, and functional programming, which makes it flexible for designing scalable applications. It allows easy integration with web frameworks and databases, making it ideal for end-to-end project development.

In addition, Python offers powerful tools for handling large datasets, performing statistical analysis, and building predictive models efficiently. The availability of opensource libraries reduces development time and effort.

- Reading the dataset
- Cleaning and preprocessing data
- Training machine learning models
- Evaluating classification performance
- Saving the final model
- Integrating the model with Flask backend

4.2 Jupyter Notebook / VS Code

Jupyter Notebook and VS Code are used as development environments. These tools help in different stages of the project such as analysis, development, testing, and deployment. Jupyter Notebook is an interactive platform mainly used for data science and machine learning tasks. It allows execution of code in small cells, making it easier

to experiment and visualize outputs step-by-step. It also supports graphs, tables, and inline explanations, which improves understanding of the data and model behavior.

VS Code (Visual Studio Code) is a powerful code editor used for building the complete project. It supports extensions, debugging tools, and integration with frameworks like Flask, making it suitable for application development and deployment.

Jupyter Notebook is useful for:

• Data analysis

• Step-by-step code execution

• Model experimentation

• Visualization

• Exploratory Data Analysis (EDA)

• Testing different algorithms and parameters VS Code is useful for:

• Writing project files

• Flask application development

• Frontend/backend integration

• Project debugging

• Managing folders and files

• Running the complete application

4.3 Flask

Flask is a lightweight Python web framework used to connect the machine learning model with the user interface. It acts as a bridge between the frontend and backend by handling requests and responses. Flask follows a simple and flexible

structure, making it easy to build web applications quickly. It allows integration of machine learning models into web-based systems for real-time predictions.

In this project, Flask is used for:

• Routing webpages

• Accepting form data

• Sending input to the ML model

• Returning prediction results• Displaying output on the result page

• Managing server-side logic

• Handling user requests efficiently

4.4 HTML

HTML (Hyper Text Markup Language) is used to create the basic structure of web pages. It defines how content is displayed on the browser. In this project, HTML is used to design forms where users can input employee details such as age, experience, salary, attendance, and other attributes required for prediction.

HTML is used for:

• Form creation

• Input fields

• Labels

• Buttons

• Result display sections

• Structuring web pages

• Organizing content clearly

4.5 CSS

CSS (Cascading Style Sheets) is used to enhance the visual appearance of the web application. It makes the interface more attractive and user-friendly. It helps in designing layouts, improving readability, and ensuring a consistent look across all pages.

CSS is used for:

• Page layout

• Colours and fonts

• Buttons and form styling

• Alignment and spacing

• Responsive design improvements

• Enhancing user interface design

4.6 JavaScript

JavaScript is used to add interactivity to the web application. It improves user experience by making the interface dynamic and responsive. It helps in validating user input before sending data to the server, reducing errors and improving efficiency.

JavaScript is used for:

• Input validation

• Dynamic UI response

• Result display enhancement

• Improved user experience

• Form validation before submission

• Providing instant feedback to users

4.7 NumPy

NumPy is a fundamental numerical computing library in Python used for handling arrays and performing mathematical operations efficiently. It provides support for multi-dimensional arrays and fast computation, which is essential for data preprocessing and model building.

NumPy is used for:

• Array operations

• Mathematical calculations

• Data transformation

• Supporting machine learning computations

• Improving computational efficiency

4.8 Pandas

Pandas is a powerful data manipulation and analysis library used for working with structured datasets. It plays a key role in this project as the dataset is stored in CSV format, and Pandas helps in reading, cleaning, and transforming the data.

Pandas is used for:

• Reading CSV files

• Cleaning data

• Handling missing values

• Selecting columns

• Encoding preparation

• Data transformation

• Filtering and sorting data

4.9 Scikit-learn

Scikit-learn is a widely used machine learning library that provides tools for data preprocessing, model training, and evaluation. It simplifies the implementation of machine learning algorithms and provides built-in functions for performance measurement.

In this project, Scikit-learn is used for:

• Logistic Regression

• Random Forest Classifier

• Train-test split

• Label encoding / one-hot encoding support

• Accuracy, precision, recall, F1-score

4.10 XGBoost

XGBoost (Extreme Gradient Boosting) is an advanced machine learning library used for high-performance classification and regression tasks. It is known for its speed and accuracy, especially when working with structured datasets.

In this project, XGBoost is used because:

• It handles structured tabular data effectively

• It provides high prediction accuracy

• It reduces overfitting using regularization techniques

• It improves model performance compared to basic algorithms

4.11 Matplotlib / Seaborn

Matplotlib and Seaborn are data visualization libraries used to create graphs and charts for better understanding of data. They help in analyzing patterns, relationships, and model performance visually.

In this project, they are used for:

• Data distribution analysis

• Correlation visualization

• Accuracy comparison graphs

• Confusion matrix plotting

4.12 Logistic Regression

Logistic Regression is a supervised learning algorithm mainly used for classification problems. It works by predicting the probability of a class using a sigmoid function. This algorithm is simple, fast, and easy to interpret. It performs well when the data is linearly separable. Logistic Regression is widely used in applications like disease prediction, spam detection, and binary classification tasks.

4.13 Random Forest Classifier

Random Forest Classifier is an ensemble learning method that combines multiple decision trees to improve prediction accuracy. Each tree is trained on a random subset of data, and their outputs are combined to make the final decision. This approach reduces over-fitting and increases model stability. It works well with both classification and regression problems. Random Forest is highly effective for handling large and complex datasets.

4.14 Train-test split

Train-test split is a technique used to divide the dataset into two parts: training data and testing data. The training data is used to build and train the machine learning model. The testing data is used to evaluate how well the model performs on unseen data. This method helps in checking the model's generalization ability. It ensures that the model is not overfit-ting the training data

4.15 Label Encoding / One-Hot Encoding

Label Encoding / One-Hot Encoding are preprocessing techniques used to convert categorical data into numerical format. Label encoding assigns a unique number to each category, which is simple but may introduce order. One-hot encoding creates separate binary columns for each category, avoiding any misleading relationships. These techniques are essential because machine learning models cannot process text data directly. Proper encoding improves model performance and accuracy.

CHAPTER 5

SYSTEM DESIGN

5.1 System Architecture

The system architecture defines how the different components of the project interact with each other to perform the overall task. The employee performance prediction system follows a layered architecture to ensure clarity, modularity, and ease of maintenance. Each layer has a specific responsibility, and data flows step-by-step from one layer to another, ensuring smooth processing and accurate prediction results. Layers in the system:

1.Data Layer – Contains the employee dataset in CSV format. This layer is responsible for storing raw data used for analysis and training.

2.Preprocessing Layer – Handles data cleaning, encoding, and feature preparation. It ensures that the data is in a suitable format for machine learning models.

3.Model Training Layer – Involves training different models such as Logistic Regression, Random Forest, and XGBoost using the prepared dataset.

4.Evaluation Layer – Compares model performance using various evaluation metrics to determine which model performs best.

5.Deployment Layer – Uses Flask to integrate the trained model with the web application for real-time predictions.

6.Presentation Layer – Includes the frontend developed using HTML, CSS, and JavaScript for user interaction.

Flow Explanation:

Dataset → Preprocessing → Train/Test Split → Model Training → Model Evaluation → Best Model Selection → Flask Integration → User Input → Prediction Output

This layered approach improves system organization, scalability, and maintainability while ensuring a clear separation of concerns.

5.2 Data Processing

Data processing is one of the most important steps in the project because machine learning models require clean, structured, and meaningful input data. Poor quality data can lead to inaccurate predictions.

The following steps are followed to prepare the dataset:

1. **Dataset Collection -**A structured employee dataset is collected in CSV format. It contains features such as employee details, performance indicators, and other relevant attributes.

2. **Removing Irrelevant Features -**Columns that do not contribute to prediction, such as Employee ID or unnecessary identifiers, are removed to reduce noise and improve model efficiency.

3. **Handling Missing Values -**Missing data is handled carefully by either filling values (using mean/median/mode) or re-moving incomplete records, depending on the situation.

4. **Categorical and Numerical Separation -**The dataset is divided into categorical and numerical features to apply appropriate preprocessing techniques for each type.

5. **Encoding Categorical Variables -**Categorical values such as job role, department, or education level are converted into numerical form using encoding techniques like label encoding or one-hot encoding.

6. **Boolean Conversion -**Boolean values such as overtime or promotion eligibility are converted into binary values (0 and 1) for easier processing.

7. **Feature Preparation -**After cleaning and transformation, the final dataset is prepared and structured properly for training machine learning models. This step ensures that the data is accurate, consistent, and ready for model training.

5.3 Machine Learning Model Design

This project uses supervised machine learning classification models because the target out-put is a predefined performance category. Different models are used to compare performance and select the best one

5.3.1 Logistic Regression

Logistic Regression is a basic classification algorithm used as a baseline model. It predicts the probability of a class and is easy to interpret. It helps in understanding how in-put features influence the prediction.

5.3.2 Random Forest Classifier

Random Forest is an ensemble learning technique that uses multiple decision trees. It improves accuracy and reduces overfitting by combining the predictions of several trees.

5.3.3 XGBoost Classifier

XGBoost is an advanced boosting algorithm that builds models sequentially. Each new model corrects the errors of the previous one, resulting in high prediction accuracy and efficiency.

5.3.4 Model Evaluation

After training, the models are evaluated using various performance metrics:

- **Accuracy:** Percentage of total predictions that the model classified correctly.
- **Precision:** Percentage of predicted positive cases that are actually correct.
- **Recall:** Percentage of actual positive cases that the model correctly identifies.
- **F1-Score:** Harmonic mean of precision and recall, balancing both metrics.
- **Confusion Matrix:** Table showing correct and incorrect predictions (TP, TN, FP, FN) of a classification model.

These metrics help in understanding how well the model performs on unseen data.

5.3.5 Final Model Selection

The model with the best performance based on evaluation metrics is selected. This model is then used for deployment in the Flask application to make real-time predictions.

5.4 Web Application Design

The web application is designed to provide a simple and user-friendly interface for interacting with the system. It eliminates the need for users to run Python code manually. The application follows a client-server architecture where the frontend collects input and the backend processes it.

Main Pages:

• Home Page / Input Form

• Prediction Result Page

Web Flow:

1. User opens the application
2. User enters employee details
3. Data is submitted to Flask backend
4. Flask processes and converts input into model format
5. The trained model predicts employee performance

5.5 Input and Output Flow

The input and output flow defines how user data is processed and how results are generated and displayed.

Input-The user provides employee-related details through the web form, such as: Age, Education, Experience, Job Role, Salary, Attendance, Training Score, Number of Projects, Overtime Job, Satisfaction.

These inputs are validated and converted into the required format before being sent to the machine learning model.

Output: The system processes the input data and displays the predicted employee performance level. The output is presented in a clear and understandable format.

Possible output categories include:

• High Performance

• Medium Performance

• Low Performance

CHAPTER 6

CODING AND IMPLEMENTATION

6.1 Importing Packages

The first step in implementation is importing all required Python libraries. These libraries provide functionalities for data handling, visualization, machine learning, and web development.

Libraries imported:

- pandas
- numpy
- matplotlib.pyplot
- seaborn
- sklearn.model_selection
- sklearn.preprocessing
- sklearn.linear_model
- sklearn.ensemble
- sklearn.metrics
- xgboost
- flask
- pickle

These libraries help in performing various tasks such as data analysis, preprocessing, model building, evaluation, and deployment. Importing the required packages at the beginning ensures that all functionalities are available throughout the project.

6.2 Data Loading

The employee dataset is loaded from a CSV file using the Pandas library. This step is essential to bring the raw data into the working environment.

Steps:

• Read CSV file using pd.read_csv()

• Display top rows using head()

• Check dataset shape (rows and columns)

• Inspect columns and datatypes

• Identify missing values

Additional checks are also performed to understand data distribution and detect inconsistencies. This step provides a clear understanding of the dataset structure before preprocessing.

6.3 Data Preprocessing

Data preprocessing ensures that the dataset is clean, consistent, and ready for model training. It plays a crucial role in improving model performance.

Steps performed:

1. Remove unwanted columns
2. Handle null values
3. Check duplicates
4. Convert boolean values
5. Handle inconsistent records

Further preprocessing may include normalization or scaling of numerical values to ensure that all features contribute equally to the model.

6.4 Feature Encoding

Most machine learning models require numeric input. Therefore, categorical variables must be converted into numerical format.

Encoding techniques:

• Label Encoding

• One-Hot Encoding (where required)

• Department → Numeric code

• Job Role → Encoded columns

• Education Level → Numeric representation

• Overtime → 0 / 1

Proper encoding ensures that categorical data is interpreted correctly by the models without introducing bias.

6.5 Model Building

After preprocessing, the dataset is prepared for model building. The data is divided into in-put and output variables.

• X → Input features

• y → Target variable (employee performance label)

Then the dataset is split into training and testing sets to evaluate model performance. Typical split:

• 80% Training

• 20% Testing

• This division helps in training the model on one part of the data and testing it on unseen data to measure accuracy.

6.6 Model Training

Different machine learning models are trained to compare their performance and select the best one.

6.6.1 Logistic Regression

• Trained as the baseline model

• Simple and efficient for classification

• Helps compare with advanced models

• Provides interpretable results

6.6.2 Random Forest Classifier

• Trained using multiple trees

• Handles non-linear feature relationships well

• Reduces overfitting through ensemble learning

• Provides better stability than single models

6.6.3 XGBoost Classifier

• Trained using boosting strategy

• Learns from mistakes of previous trees

• Often provides better classification accuracy

• Efficient for handling complex datasets

6.7 Model Evaluation

After training, each model is evaluated using different performance metrics to determine its effectiveness.

Evaluation metrics:

• Accuracy Score

• Precision Score

• Recall Score

• F1-Score

• Confusion Matrix

To identify which model provides:

• Best prediction accuracy

• Balanced precision and recall

• Reliable class separation Final result

The best-performing model is selected and saved for deployment. This ensures that the most accurate and reliable model is used in the application.

6.8 Flask Integration

After selecting the best model, it is integrated into a web application using Flask.

Steps:

1. Save model using pickle
2. Create app.py file
3. Load model in Flask
4. Create routes for:
5. Receive form input
6. Convert input into model-ready format

7. Predict performance

8. Return result to HTML page

Flask acts as a bridge between the frontend and backend, enabling real-time predictions based on user input.

6.9 Frontend Implementation

The frontend is implemented to provide a user-friendly interface for interacting with the system.

Technologies used:

• HTML for structure

• CSS for styling

• JavaScript for validation Frontend features:

• User-friendly form

• Input labels

• Drop-downs / text fields / number fields

• Submit button

• Result page

• Clear layout and responsive design

CHAPTER 7

SYSTEM TESTING

7.1 Introduction

Testing is an important phase of the project that ensures the system works correctly, reliably, and efficiently. The employee performance prediction system is tested at both the ma-chine learning model level and the web application level.

Proper testing helps in identifying errors, improving system performance, and ensuring that the final application meets user requirements. It also ensures that the system behaves as expected under different conditions and inputs.

7.2 Testing and Validation

The project is validated in two main ways:

1. Model Validation

• Checking classification performance on test data

• Comparing multiple models

• Verifying prediction accuracy and consistency

• Ensuring the model generalizes well to unseen data

2. Application Validation

• Testing the Flask application

• Verifying form submission and output display

• Ensuring proper communication between frontend and backend

• Checking real-time prediction functionality

This dual validation ensures both the correctness of the model and the usability of the application.

7.3 Unit Testing

Unit testing focuses on testing individual components or functions independently to ensure they work correctly.

Examples:

• Dataset loading function

• Preprocessing function

• Encoding function

• Prediction function

• Model loading function

Each module is tested separately to detect errors early and ensure that every part of the system performs its specific task correctly.

7.4 Integration Testing

Integration testing verifies whether different modules of the system work together properly.

Examples:

• Frontend form → Flask route

• Flask route → Model input conversion

• Model prediction → Output rendering

This testing ensures smooth data flow between components and confirms that the system functions correctly when modules are combined.

7.5 Functional Testing

Functional testing checks whether the system performs all intended functions as per requirements.

Checked items:

• Input form loads properly

• All fields accept valid values

• Submit button works correctly

• Backend receives and processes data

• Prediction result is displayed correctly

• Error messages are shown for invalid inputs

This ensures that the application meets all functional requirements.

7.6 System Testing

System testing evaluates the complete system as a whole to ensure it works correctly in real-world scenarios.

Verified:

• Full workflow from user input to prediction output

• No crashes during normal usage

• Stable performance with multiple test entries

• Proper handling of edge cases and invalid inputs

This testing confirms that the system is ready for deployment and real-time usage.

7.7 Black Box Testing

In black box testing, the internal implementation is not considered. Only input and output behaviour is tested.

Examples:

• Input valid employee details → Expect performance prediction

• Leave required fields blank → Expect validation message

• Enter invalid data → Expect controlled error or prevention

• Provide extreme values → System should still respond correctly

This ensures that the system behaves correctly from the user's perspective.

7.8 Test Cases and Validation Results

TestCase ID	Test Scenario	Input	Expected Output	Result
TC01	Open application	URL	Home page loads	Pass
TC02	Submit valid employee details	Valid form data	Prediction displayed	Pass
TC03	Submit empty required field	Missing input	Validation error	Pass
TC04	Enter invalid numeric field	Text in numeric field	Error / blocked	Pass
TC05	Backend prediction route	Valid POST request	Result returned	Pass
TC06	Model output check	Encoded data	Performance class shown	Pass
TC07	Multiple submissions	Different inputs	Consistent results	Pass
TC08	Edge case input	Extreme values	Valid prediction / handled	Pass

Table 7.8 Test Cases and Validation Results

CHAPTER 8

SCREENSHOTS

8.1 Title and Description Page

Figure 8.1 – Title and Description Page

This screenshot represents the introductory page of the web application, which provides an overview of the Employee Performance Prediction system. It includes the project title and a brief description explaining the purpose and functionality of the application. The page highlights how machine learning models are used to analyze employee data and predict performance levels.

8.2 Single Employee Input Page

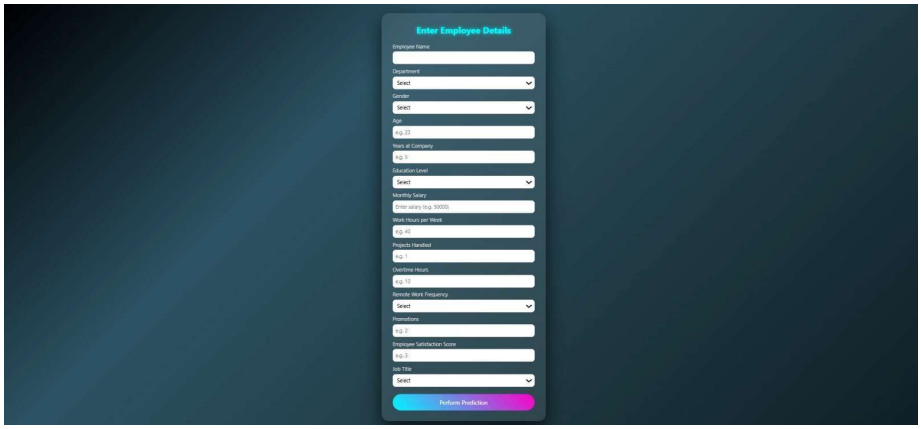

Figure 8.2 – Single Employee Input Page

This screenshot shows the page where users can enter details of a single employee for performance prediction. The interface is designed to be simple, structured,

and user-friendly, allowing easy data entry. The form includes fields such as age, education, job role, experience, salary, attendance, training score, number of projects, overtime, and job satisfaction. Each field is clearly labeled to guide the user in providing accurate information.

8.3 Single Prediction Result Page

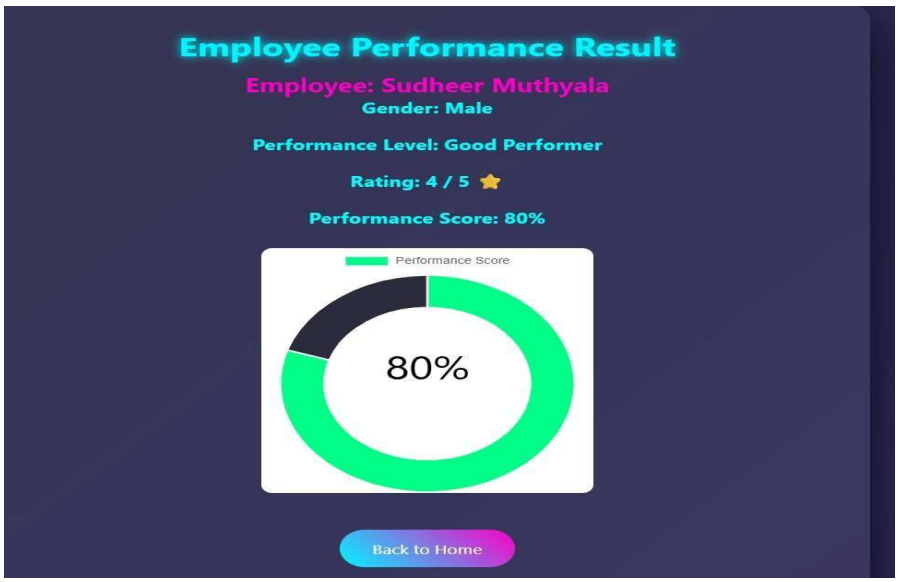

Figure 8.3 – Single Prediction Result Page

This screenshot displays the result of the prediction for a single employee after submitting the input form. The system processes the entered data using trained machine learning models and presents the output in a clear and understandable format. The result typically indicates performance levels such as “High Performance” or “Low Performance.” The page is designed to highlight the result using formatted text or visual emphasis, making it easy for users to interpret. It may also include a summary of the input data to provide context for the prediction. ingle Prediction Result Page .

8.4 Multiple Employee Input Page

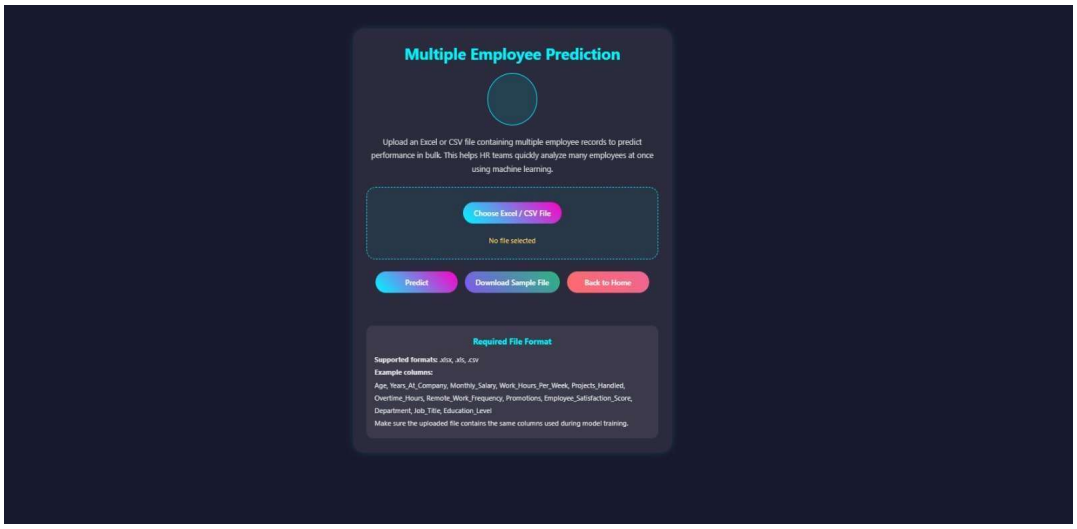

Figure 8.4 – Multiple Employee Input Page

This screenshot represents the multiple employee input page where users can upload a CSV or Excel file for bulk prediction. The interface is simple and user-friendly, with a clear option to choose and upload a file. It also provides guidance on uploading employee data for analysis using machine learning. Buttons like “Predict,” “Download Sample File,” and “Back to Home” improve usability.

8.5 Multiple Employee Result Page

multiple result

Figure 8.5 – Multiple Employee Input Page

This screenshot represents the multiple employee prediction result page, where the system displays results after processing the uploaded CSV file. It shows a confirmation message indicating that the prediction process has been completed

successfully. The page includes a summary with details such as file name, number of employees processed, and prediction status. Results are categorized into high, medium, and low performers with their counts clearly shown. It also provides options to download the results, go back to the upload page, or return to the home page.

8.6 Flask Backend Output

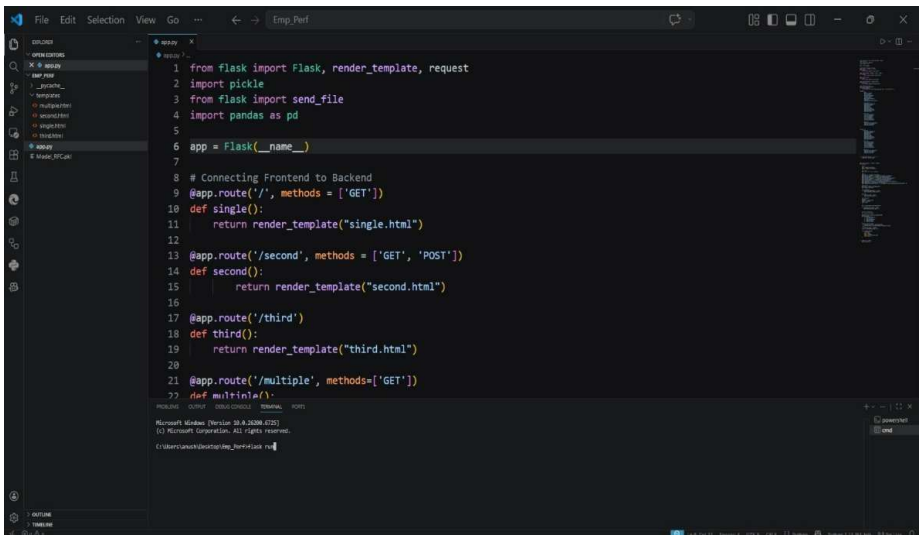

Figure 8.6 – Flask Server Output

This screenshot shows the Flask backend running in the terminal or command prompt. It displays important server information such as active routes, incoming requests, and response status. Messages like “Running on http://127.0.0.1:5000/” confirm that the server is running successfully. The backend is responsible for handling form submissions, processing CSV file uploads, loading trained machine learning models (such as Logistic Regression, Random Forest, and XGBoost), and generating predictions. development.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENT

9.1 Conclusion

This project, "A Machine Learning Approach to Employee Performance Prediction," successfully demonstrates how machine learning techniques can be applied to improve employee evaluation and support HR decision-making processes. The system analyzes historical employee data and provides predictions in a faster, more objective, and more consistent manner compared to traditional manual appraisal methods.

The project implements three supervised classification algorithms: Logistic Regression, Random Forest Classifier, and XGBoost Classifier. These models are trained on pre-processed employee data and evaluated using performance metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. By comparing multiple models, the system effectively identifies the most suitable algorithm for predicting employee performance.

The integration of a web application using Flask, HTML, CSS, and JavaScript enhances the practical usability of the system. It allows users to input employee details through a simple interface and instantly receive prediction results, making the system accessible even to non-technical users.

In addition, the project highlights the importance of data preprocessing, feature selection, and model evaluation in achieving reliable results. The system demonstrates how data-driven approaches can reduce human bias and improve decision-making accuracy.

Overall, this project proves that machine learning can significantly improve the efficiency, consistency, and usefulness of employee performance evaluation. It can assist HR professionals and managers in identifying performance trends, recognizing high-performing employees, and taking proactive actions to improve workforce productivity.

9.2 Future Enhancement

Although the project is successful, several improvements can be made in the future to enhance its performance, scalability, and usability:

1. **Use larger real-world HR datasets -**Using more diverse and large-scale datasets can improve model accuracy and generalization across different organizations.

2. **Add more employee behavioural features -**Additional features such as peer feedback, project deadlines, learning growth, communication skills, and productivity trends can provide deeper insights into employee performance.

3. **Hyperparameter tuning -**Advanced optimization techniques can be applied to models like Random Forest and XGBoost to achieve better performance and accuracy.

4. **Class imbalance handling -**Techniques such as SMOTE or resampling methods can be used to handle imbalanced datasets and improve model fairness.

5. **Explainable AI integration -**Tools like SHAP or feature importance visualization can be added to explain how predictions are made, increasing transparency and trust in the system.

6. **Role-specific models -**Separate models can be developed for different departments or job roles to improve prediction relevance and accuracy.

7. **Database integration -**The system can be connected to databases such as MySQL or enterprise HR systems to store and manage large volumes of employee data efficiently.

8. **Cloud deployment -**Deploying the application on cloud platforms (like AWS, Azure, or Google Cloud) will allow remote access and scalability.

CHAPTER 10

REFERENCES

10.1 Research Papers

1. Kirimi, J. M., & Moturi, C. A. – Studies related to employee performance prediction using classification techniques.
2. Lather, A. S., et al. – Comparative studies on supervised machine learning algorithms for workforce analytics.
3. Mahmoud, et al. – Employee appraisal and hiring-based performance prediction using data-driven approaches.
4. Pathak, et al. (2023) – Advanced machine learning approaches for employee performance classification.
5. Research papers on HR analytics and employee productivity prediction using supervised learning.
6. Studies on ensemble learning methods in employee-related classification tasks.

10.2 Websites

1. https://www.kaggle.com

For employee datasets and project references.

2.https://scikit-learn.org

Official documentation for machine learning algorithms and evaluation metrics.

3. https://xgboost.readthedocs.io

Official documentation for XGBoost Classifier.

4. https://pandas.pydata.org

Official documentation for data manipulation.

5. https://numpy.org

Official documentation for numerical computation.

6. https://matplotlib.org

Official documentation for data visualization.

7. https://seaborn.pydata.org

Statistical visualization library documentation.

8. https://flask.palletsprojects.com

Official documentation for Flask framework.

9. https://developer.mozilla.org

Reference for HTML, CSS, and JavaScript.

ANNEXURE

SOURCE CODE

A.1 Frontend Implementation

The frontend of the system is designed using HTML, CSS, and JavaScript. It provides user interfaces for single employee prediction, multiple employee prediction, and result visualization.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Employee Performance Prediction</title>
<!-- ================= CSS STYLING ================= -->

<style>
/* Reset default styling */* {   margin: 0;   padding: 0;   box-sizing: border-box;   font-
family: "Segoe UI", Arial, sans-serif; } /* Background and overall layout */ body {
min-height: 100vh;   background: linear-gradient(-45deg, #0f0c29, #302b63,
#24243e, #000000);   color: white;   display: flex;   justify-content: center;   align-
items: center; }
/* Main container */.container {   width: 90%;   max-width: 800px;   padding: 25px;
border-radius: 20px;   background: rgba(255, 255, 255, 0.08);   text-align: center; } /*
Heading style */h1 {   color: #00f7ff;   margin-bottom: 15px; } /* Buttons */ button {
padding: 10px 20px;   margin: 10px;   border: none;   border-radius: 25px;
background: linear-gradient(45deg, #00f7ff, #ff00c8);   color: white;   cursor: pointer;
} /* Form inputs */ input, select {   width: 100%;   padding: 8px;   margin: 8px 0;
border-radius: 8px;
  border: none;
 }
  </style>
</head>
<body>
<div class="container">
```

```
<!-- ================= LANDING PAGE ================= -->
<h1>Employee Performance Prediction</h1>
<p>
This system predicts employee performance using machine learning based on input features like age, salary, experience, and job role.
</p>
<button onclick="showForm()">Single Prediction</button>
<button onclick="showUpload()">Multiple Prediction</button>
<!-- ================= SINGLE EMPLOYEE FORM ================= -->
<div id="formSection" style="display:none; margin-top:20px;">
<h2>Enter Employee Details</h2>
<form action="/predict" method="POST">
<!-- Employee basic details -->
<input type="text" name="name" placeholder="Employee Name" required>
<select name="department">
  <option>IT</option>
  <option>HR</option>
  <option>Finance</option>
</select>
<input type="number" name="Age" placeholder="Age" required>
<input type="number" name="Monthly_Salary" placeholder="Salary" required>
<input type="number" name="Work_Hours_Per_Week" placeholder="Work Hours" required>
<button type="submit">Predict</button>
</form>
</div>
<!-- =================MULTIPLE FILE UPLOAD =================
<div id="uploadSection" style="display:none; margin-top:20px;">
<h2>Upload Employee File</h2>
<input type="file" id="fileInput" accept=".csv,.xlsx">
<button onclick="uploadFile()">Upload & Predict</button>
<p id="status"></p>
</div>
```

```
<!-- ================= RESULT DISPLAY ================= -->
<div id="resultSection" style="display:none; margin-top:20px;">
<h2>Prediction Result</h2>
<p id="resultText"></p>
</div>
</div>
<!-- ================= JAVASCRIPT LOGIC ================= -->
<script>
/* Show single prediction form */ function showForm() {
document.getElementById("formSection").style.display = "block";
document.getElementById("uploadSection").style.display = "none";
 }
/* Show file upload section */ function showUpload() {
document.getElementById("uploadSection").style.display = "block";
document.getElementById("formSection").style.display = "none";
}
/* Upload file and send to backend */ function uploadFile() {   let file =
document.getElementById("fileInput").files[0];
  if (!file) {
    document.getElementById("status").innerText = "Please select a file!";
    return;
  }
  let formData = new FormData();   formData.append("file", file);
fetch("/predict_multiple", {     method: "POST",     body: formData
  })
  .then(res => res.blob())   .then(data => {
document.getElementById("status").innerText =       "Prediction completed and file
downloaded!";   })
 .catch(() => {     document.getElementById("status").innerText =    "Error in
prediction process.";
  });
}
</script>
</body>
```

```
</html>
```

A.2 Backend Implementation

The backend is developed using Flask framework, which handles user requests, pro-cesses input data, integrates the machine learning model, and returns prediction results.

```
# IMPORTING REQUIRED LIBRARIES

from flask import Flask, render_template, request, send_file     # Flask modules for
web app import pickle  # Used to load trained ML model import pandas as pd        #
Used for data handling and DataFrame creation
# INITIALIZING FLASK APPLICATION
app = Flask(__name__)      # Create Flask app instance
# ROUTING (FRONTEND CONNECTION)
@app.route('/', methods=['GET']) def home():
 # Load landing page (single.html)     return render_template("single.html")
@app.route('/second', methods=['GET', 'POST']) def second():
  # Load employee input form page   return render_template("second.html")
  @app.route('/third') def third():     # Load result page     return
render_template("third.html") @app.route('/multiple', methods=['GET']) def
multiple():
 # Load multiple prediction upload page     return render_template("multiple.html")
# DOWNLOAD SAMPLE FILE
@app.route('/download_sample') def download_sample():
# Send sample Excel file to user send_file('static/sample_emp_data.xlsx',
as_attachment=True)
# FEATURE COLUMNS USED IN MODEL
columns = [
    'Age', 'Years_At_Company', 'Monthly_Salary',
    'Work_Hours_Per_Week', 'Projects_Handled',
    'Overtime_Hours', 'Remote_Work_Frequency',
    'Promotions', 'Employee_Satisfaction_Score',
    # Department (One-Hot Encoded)
```

```
    'Department_Engineering', 'Department_Finance',
    'Department_HR', 'Department_IT', 'Department_Legal',
    'Department_Marketing', 'Department_Operations',
   'Department_Sales',
    # Job Title
    'Job_Title_Consultant', 'Job_Title_Developer',
    'Job_Title_Engineer', 'Job_Title_Manager',
    'Job_Title_Specialist', 'Job_Title_Technician',
    # Education Level
    'Education_Level_High School',
    'Education_Level_Master',
    'Education_Level_PhD'
]
# LOAD TRAINED MACHINE LEARNING MODEL
with open("Model_RFC.pkl", "rb") as f:     model_rfc = pickle.load(f)   # Load Random
Forest model
# SINGLE EMPLOYEE PREDICTION
@app.route('/predict', methods=["POST"]) def predict():
    # Get basic details from form     name = request.form["name"]     gender =
request.form["gender"]     # Initialize all features with 0     data = {col: 0 for col in
columns}     # NUMERICAL INPUT DATA
    data['Age'] = int(request.form["Age"]   data['Years_At_Company'] =
int(request.form["Years_At_Company"])  data['Monthly_Salary'] =
int(request.form["Monthly_Salary"])     data['Work_Hours_Per_Week'] =
int(request.form["Work_Hours_Per_Week"])     data['Projects_Handled'] =
int(request.form["Projects_Handled"])     data['Overtime_Hours'] =
int(request.form["Overtime_Hours"])     data['Remote_Work_Frequency'] =
int(request.form["Remote_Work_Frequency"])     data['Promotions'] =
int(request.form["Promotions"])     data['Employee_Satisfaction_Score'] = float(
request.form.get("Employee_Satisfaction_Score", 0)
    )
```

```
# CATEGORICAL DATA (ONE-HOT ENCODING)
    department = request.form["department"]     job = request.form["job"]     # Set
selected department to 1     if f"Department_{department}" in data:
data[f"Department_{department}"] = 1
    # Set selected job role to 1     if f"Job_Title_{job}" in data:
data[f"Job_Title_{job}"] = 1     # Map education levels     edu_map = {
        "Bachelor": "High School",
        "Master": "Master",
        "PhD": "PhD"
}     edu = edu_map[request.form["education"]]     if f"Education_Level_{edu}" in
data:        data[f"Education_Level_{edu}"] = 1
    # CONVERT TO DATAFRAME
    df = pd.DataFrame([data])   # Convert dictionary to DataFrame
    # MODEL PREDICTION
    prediction_value = model_rfc.predict(df)[0]   # Get prediction
    # Mapping numeric output to labels     performance_map = {
        1: "Very Low Performer",
        2: "Low Performer",
        3: "Average Performer",
        4: "Good Performer",
        5: "High Performer"
    }
    # Convert prediction to label prediction_label =
performance_map.get(int(prediction_value), "Unknown")
    # Convert to percentage score     score = int(prediction_value) * 20
    # RETURN RESULT TO FRONTEND
    return render_template(
   "third.html",        name=name,     gender=gender,   prediction=prediction_label,
score=score }
# RUN THE APPLICATION
if __name__ == "__main__":
    app.run(debug=True)   # Run Flask app in debug mode
```

A. 3 Machine Learning Model Code

The machine learning model is developed using Python libraries such as Scikit-learn and XGBoost. It is trained on employee data to predict performance levels.

```
# IMPORTING REQUIRED LIBRARIES

import numpy as np     # Used for numerical operations
 import pandas as pd    # Used for data manipulation and analysis import
matplotlib.pyplot as plt    # Used for plotting graphs import seaborn as sns    # Used
for advanced data visualization
# Machine Learning Libraries from sklearn.model_selection import train_test_split   #
To split dataset into training and testing sets from sklearn.preprocessing import
LabelEncoder  # To convert categorical data into numeri-cal form from
sklearn.preprocessing import StandardScaler
 # To scale the feature values from sklearn.metrics import accuracy_score            # To
calculate model accuracy from sklearn.metrics import classification_report      # To
get precision, recall, F1score
from sklearn.metrics import confusion_matrix        # To create confusion matrix

# ML Models from sklearn.linear_model import LogisticRegression
 # Logistic Regression model from sklearn.ensemble import RandomForestClassifier
# Random Forest model from xgboost import XGBClassifier                    # XGBoost
model
# LOADING THE DATASET

data = pd.read_csv("employee_data.csv")   # Load dataset from CSV file

print(data.head())                    # Display first 5 rows of dataset

    # DATA PREPROCESSING

data = data.dropna()                    # Remove missing (null) values from dataset #
Convert categorical columns into numeric using Label Encoding label_encoder =
LabelEncoder()           # Create LabelEncoder object for column in
```

```
data.select_dtypes(include=['object']).columns:   # Loop through categorical columns
data[column] = label_encoder.fit_transform(data[column])    # Encode each column
# FEATURE & TARGET SEPARATION

X = data.drop("Performance", axis=1)          # Features (independent variables) y =
data["Performance"]   # Target variable (dependent variable)
 # FEATURE SCALING scaler = StandardScaler()                # Create scaler object
X = scaler.fit_transform(X)              # Scale features to standard range

# TRAIN-TEST SPLIT

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_state=42
)     # Split data (80% train, 20% test)

  # LOGISTIC REGRESSION MODEL

lr_model = LogisticRegression()          # Initialize Logistic Regression model
lr_model.fit(X_train, y_train)           # Train model using training data y_pred_lr =
lr_model.predict(X_test)      # Predict test data print
("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))
  #Print accuracy
print(classification_report(y_test, y_pred_lr))            # Print detailed report

  # RANDOM FOREST CLASSIFIER

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)

# Initialize Random Forest

rf_model.fit(X_train, y_train)                                # Train model y_pred_rf =
rf_model.predict(X_test)                              # Predict test data print("Random Forest
Accuracy:", accuracy_score(y_test, y_pred_rf))  # Print accuracy
print(classification_report(y_test, y_pred_rf))                    # Print report-

# XGBOOST CLASSIFIER

xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='mlogloss')    #
Initialize XGBoost
```

```
xgb_model.fit(X_train, y_train)                                  # Train model y_pred_xgb
= xgb_model.predict(X_test)                                # Predict test data
print("XGBoost Accuracy:", accuracy_score(y_test, y_pred_xgb))     # Print accuracy
print(classification_report(y_test, y_pred_xgb))                     # Print report
# CONFUSION MATRIX VISUALIZATION

plt.figure(figsize=(6, 4))                          # Set figure size
sns.heatmap(confusion_matrix(y_test, y_pred_rf)  # Create heatmap of confusion
matrix            annot=True, fmt='d')                     # Show numbers inside cells
plt.title("Confusion Matrix - Random Forest")          # Title of the graph
 plt.xlabel("Predicted")                            # X-axis label plt.ylabel("Actual")
# Y-axis label plt.show()                                     # Display plot
# SAVING THE TRAINED MODEL
import joblib                                  # Library to save model
joblib.dump(rf_model, "employee_performance_model.pkl") # Save trained Random
Forest model

# PREDICTION ON NEW DATA
new_data = X_test[0].reshape(1, -1)    # Take one sample from test data and reshape it
prediction = rf_model.predict(new_data)  # Predict performance for new data
print("Predicted Performance:", prediction)  # Display prediction result
```

A. 4 Model Saving & Integration

Model integration is the process of connecting the trained machine learning model with the backend system. The saved model is loaded using pickle and used to generate pre-dictions based on user input data.

```
# IMPORT REQUIRED LIBRARIES
import pickle              # Used to load saved machine learning model import pandas as
pd        # Used to handle input data
# LOAD TRAINED MODEL
# Open the saved Random Forest model file (.pkl)
with open("Model_RFC.pkl", "rb") as file:
    model = pickle.load(file)   # Load model into memory
# DEFINE INPUT FEATURES
```

```
# These are the same features used during model training columns = [
  'Age', 'Years_At_Company', 'Monthly_Salary',
    'Work_Hours_Per_Week', 'Projects_Handled',
    'Overtime_Hours', 'Remote_Work_Frequency',
    'Promotions', 'Employee_Satisfaction_Score',
    # Department (one-hot encoded)
    'Department_Engineering', 'Department_Finance',
    'Department_HR', 'Department_IT', 'Department_Legal',
    'Department_Marketing', 'Department_Operations',
    'Department_Sales',
    # Job roles
    'Job_Title_Consultant', 'Job_Title_Developer',
    'Job_Title_Engineer', 'Job_Title_Manager',
    'Job_Title_Specialist', 'Job_Title_Technician',
    # Education levels
    'Education_Level_High School',
    'Education_Level_Master',
    'Education_Level_PhD'

]

# SAMPLE INPUT DATA (FROM USER)

# Initialize all features with default value 0 data = {col: 0 for col in columns} # Assign
values (example input) data['Age'] = 30 data['Years_At_Company'] = 5
data['Monthly_Salary'] = 50000 data['Work_Hours_Per_Week'] = 40
data['Projects_Handled'] = 3 data['Overtime_Hours'] = 5
data['Remote_Work_Frequency'] = 50 data['Promotions'] = 1
data['Employee_Satisfaction_Score'] = 4.2 # Set categorical values (one-hot
encoding) data['Department_IT'] = 1 data['Job_Title_Developer'] = 1
data['Education_Level_Master'] = 1
  # CONVERT INPUT TO DATAFRAME
# Convert dictionary into DataFrame format (required by model) df =
pd.DataFrame([data])
# MAKE PREDICTION
```

```
# Use loaded model to predict performance prediction_value = model.predict(df)[0]
# CONVERT OUTPUT TO LABEL
# Mapping numeric prediction to human-readable output performance_map = {
    1: "Very Low Performer",
    2: "Low Performer",
    3: "Average Performer",
    4: "Good Performer",
    5: "High Performer"
}
performance_map.get(int(prediction_value), "Unknown")
# Convert prediction into percentage score score = int(prediction_value) * 20
# DISPLAY RESULT
 print("Predicted Performance:", prediction_label)
print("Performance Score:", score, "%")
```